Interpretable Control for Unstable Systems via SINDy-RL

Patrick Smith · Andrew Falcone

ME 595 · University of Washington · Spring 2026

What if the controller was just an equation?

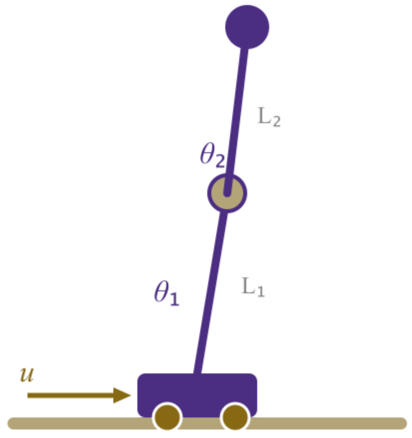
$$u(x) = \underbrace{-2.4\,\theta_1}_{\text{balance}} - \underbrace{0.9\,\theta_2}_{\text{balance}} - \underbrace{0.5\,\dot{\theta}_1}_{\text{damp}} - \underbrace{0.2\,\dot{\theta}_2}_{\text{damp}} + \cdots$$

8 terms · every coefficient has a physical interpretation · fits on a napkin

The Inverted Double Pendulum

State $x \in \mathbb{R}^6$

$$x = \begin{bmatrix} x_{\text{cart}} \\ \theta_1 \\ \theta_2 \\ \dot{x}_{\text{cart}} \\ \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix}$$



Action $u \in [-1,\ 1]$ — cart force

Unstable equilibrium at $\theta_1 = \theta_2 = 0$

$L_1 = L_2 = 0.6$ m · dt = 0.05 s · MuJoCo physics

SINDy — Discovering equations from data

Key idea: dynamics live in a low-dimensional function space.

$$\dot{X} = \underbrace{\Theta(X, U)}_{\text{candidate feature library}} \cdot \underbrace{\Xi}_{\text{sparse coefficients}}$$

$$\Theta = \left[\, 1 \mid x \mid \theta_1,\ \theta_2 \mid x^2,\ x\theta_1,\ \theta_1^2,\ \ldots \,\right] \quad \text{degree-}d \text{ polynomial library}$$

STLSQ solver drives most coefficients to *exactly zero* — exposing the true governing terms.

Library degree *d* is a design choice — it must match the complexity of the system and is not obvious a priori.

Lineage: LASSO '96 → SINDy '16 → SINDy-C '18 → E-SINDy '22 → **SINDy-RL '24** | Koopman as the competing paradigm

Two objectives — one interpretable controller

① REDUCED ORDER MODEL

$$X' = \Theta(X,U)\Xi$$

SINDy dynamics surrogate
enables simulator-free RL training

PILCO → PETS → DreamerV3 → SINDy-RL

train RL
in surrogate

NN Policy

$$u = \pi(x)$$

distill

② SPARSE DICTIONARY POLICY

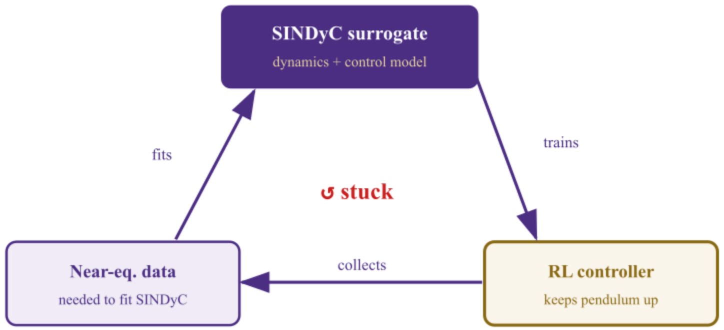
$$u \approx \Theta(x)\xi$$

interpretable polynomial control law
distilled from the trained NN policy

ALVINN → DAgger → augmented BC

SINDyC is a chicken-and-egg problem

You can't train near equilibrium without reaching it — and you can't reach it without a controller.



> Solution: co-train the controller and surrogate in an iterative active-learning loop.

Baseline — the performance ceiling

A standard PPO agent trained with **full simulator access**.

100%
Task success rate
(20/20 eval episodes)

9359
Mean reward
(max possible ≈ 10,000)

9,731
Policy parameters
MLP [64 × 64]

50,000 transitions collected from the trained policy — the dataset used for sparse learning.

Sparse policy distillation — the compounding error trap

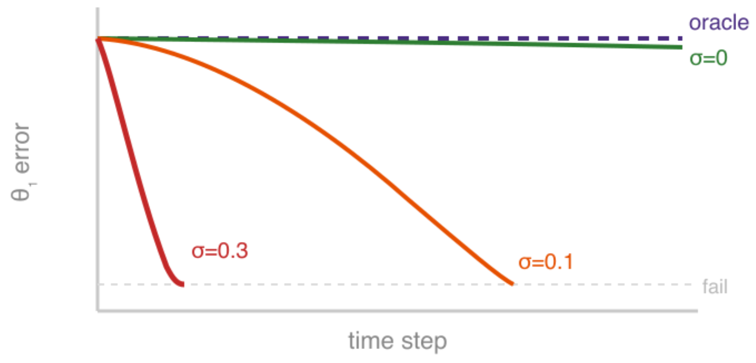
Behavioral cloning on the oracle dataset:

$$\min_{\Xi} \left\|\Theta(X)\,\Xi - U^*\right\|_2 \;+\; \lambda\|\Xi\|_1$$

Degree-4 library: 210 terms → STLSQ selects **8 terms ✓**

But the policy fails in deployment.

At noise σ = 0.3 rad: **1000 steps → ~20 steps**



Noise σ	Mean episode length
0 (training)	~1000 steps ✓
0.1 rad	~200 steps
0.3 rad	~20 steps ✗

> Off-distribution states produce small action errors → errors compound → catastrophic failure.

The fix: query the oracle, for free

Key insight: the NN policy is a pure function of state — no memory, no rollouts needed. Query it at *any* state we choose.

For each of 3 rounds:

1. Perturb states: $\tilde{x} = x + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0,\, 0.15^2)$
2. Query oracle: $\tilde{u} = \pi_{\mathrm{NN}}(\tilde{x})$
3. Append $(\tilde{x}, \tilde{u})$ to dataset

Dataset grows **4×** (50k → 200k pairs). STLSQ re-fit recovers cross-coupling terms.

	σ = 0.1	σ = 0.3
Base policy	~200 steps	~20 steps ✗
Augmented	~1000 steps ✓	~500–900 steps

Baseline NN: 1000 steps at all noise levels

25–45× more robust — zero additional simulator interactions.

$R^2 \approx 0.97$ ceiling: Tanh NN ≠ polynomial — structural mismatch.
✓ **Polynomial actor**: degree-2 library, 44→22 terms (STLSQ) → $R^2 = 0.999$, 1000/1000 steps at all noise.

ROM surrogate — train RL inside an interpretable model

Core idea: use a SINDy surrogate as the "dream environment" (cf. DreamerV3).

Bootstrap PD + random → Fit SINDy polynomial dynamics → Train PPO in surrogate → Deploy in real sim → Collect data near equilibrium →

⟳

DreamerV3 concept	This project
Latent world model (RSSM)	SINDy surrogate — explicit, interpretable
"Dreaming" (policy training)	PPO in `SINDySurrogateEnv`
World model update	Refit SINDy on expanded dataset
Real env interaction	Controller rollout in MuJoCo

ROM surrogate — results

Iterative RMSE convergence:

Iteration	One-step RMSE	Real sim steps
0 (bootstrap)	—	~3,000
1	Δ large	+~5,000
2	Δ small	+~5,000

Each iteration trains a better policy which collects better data.

> Baseline NN required **400,000** real-sim steps.
> ROM surrogate target: **< 50,000** — 8× more data-efficient.

Surrogate environment — exact reward replica:

$$r = 10 - \left(0.01\, x_{\text{tip}}^2 + (y_{\text{tip}} - 2)^2\right) - v_{\text{pen}}$$

$$y_{\text{tip}} = L_1 \cos\theta_1 + L_2 \cos(\theta_1 + \theta_2)$$

The SINDy model is **interpretable dynamics**:

$$x_{k+1} = x_k + \Delta t \cdot \Xi^T \Theta(x_k, u_k)$$

How do they compare?

Approach	Real-sim steps	Policy type	Mean length	Success	Interpretable
Baseline NN	400,000	NN (9,731 params)	1,000	100%	✗
Sparse policy (base)	400,000	Polynomial (8 terms)	~20 @ σ=0.3	Low	✓
Sparse policy (augmented)	400,000*	Polynomial	~500–900	~50–90%	✓
ROM surrogate RL	~50,000 (est.)	NN in surrogate	TBD	TBD	◑
Phase 3 (stretch)	TBD	Polynomial	—	—	✓

* No additional real-sim steps beyond baseline training — augmentation uses oracle queries only.

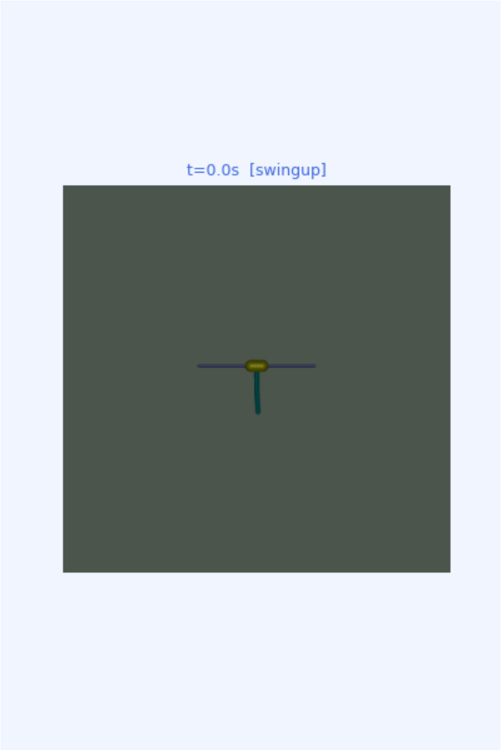
Stretch Goal

PHASE 3 · FULLY INTERPRETABLE CLOSED-LOOP CONTROL

Combine the interpretable **dynamics** (ROM) with the interpretable **policy** (sparse dictionary)

Bonus

Make it works from down initial position



Currently: Swing-up PPO → handoff → Stabilizer PPO · 304 steps (15.2 s) · **SUCCESS**

Interpretable control for unstable systems.

It works.

Patrick Smith · **Andrew Falcone**

ME 595 · University of Washington · Spring 2026